МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное учреждение
высшего образования «Крымский федеральный университет
имени В.И. Вернадского»
Физико-технический институт (структурное подразделение)
Кафедра компьютерной инженерии и моделирования

Учебно-методическое пособие
«Компьютерная визуализация пирамидальной сортировки»
по учебной дисциплине
«Структуры и алгоритмы обработки данных»

для направлений подготовки
09.03.01 "Информатика и вычислительная техника",
09.03.04 «Программная инженерия»

квалификация бакалавр

Симферополь 2023

Учебно-методическое пособие «Компьютерная визуализация пирамидальной сортировки» по учебной дисциплине «Структуры и алгоритмы обработки данных». – Симферополь: ФГАОУ ВО «Крымский федеральный университет имени В. И. Вернадского», 2023. – 24 стр.

Разработчики - Горская И.Ю., старший преподаватель кафедры компьютерной инженерии и моделирования, Жибер Е.А., студент 2 курса направления подготовки 09.03.04 «Программная инженерия»

Утверждено на заседании кафедры компьютерной инженерии и моделирования, протокол № 6 от 25.05.2023г.

Заведующий кафедрой компьютерной инженерии и моделирования  (Милюков В.В.)

Издается по решению Методического совета Физико-технического института ФГАОУ ВО «КФУ им. В.И. Вернадского»
(протокол №9 от 20.06.2023 г.).

Председатель методической комиссии  (А.Ф. Рыбась)

© Горская И.Ю., Жибер Е.А., 2023
© ФГАОУ ВО «Крымский федеральный университет имени В.И. Вернадского», 2023

1. Теоретические сведения

1.1. Постановка задачи сортировки

Пожалуй, никакая другая проблема не породила такого количества разнообразнейших решений, как задача сортировки. Приведем ее формальную постановку.

Задача сортировки. Пусть есть последовательность a_0, a_1... a_{n-1} и функция сравнения, которая на любых двух элементах последовательности принимает одно из трех значений: меньше, больше или равно. Требуется переставить члены последовательности таким образом, чтобы выполнялось условие: $a_i \le a_{i+1}$, для всех i от 0 до $n - 1$.

Возможна ситуация, когда элементы состоят из нескольких полей, например, так, как показано на рис. 1.1.

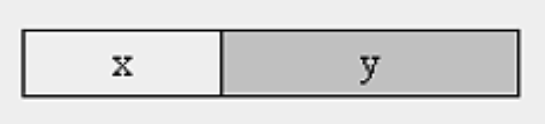


Рис. 1.1. Структура элемента последовательности

Если значение функции сравнения зависит только от поля x, то x называют ключом, по которому производится сортировка. На практике, в качестве x часто выступает число, а поле y хранит какие-либо данные, никак не влияющие на работу алгоритма.

Существует ли некий "универсальный", наилучший алгоритм? Вообще говоря, нет. Однако, имея приблизительные характеристики входных данных, можно подобрать метод, работающий оптимальным образом.

Ниже мы рассмотрим пирамидальную сортировку, и проведем сравнительный анализ ее с другими алгоритмами. Для обоснованного выбора метода решения, рассмотрим параметры, по которым производится оценка алгоритмов сортировки.

1. *Временная сложность* сортировки – основной параметр, характеризующий быстродействие алгоритма.

2. *Память* – ряд алгоритмов требует выделения дополнительной памяти под временное хранение данных. При оценке используемой памяти не будет учитываться место, которое занимает исходный массив и независящие от входной последовательности затраты, например, на хранение кода программы.

3. *Устойчивость* – устойчивая сортировка не меняет взаимного расположения равных элементов. Такое свойство может быть очень полезным, если они состоят из нескольких полей, как на рис. 1.1, а сортировка происходит по одному из них, например, по *Key*. Свойство устойчивости удобно пояснить конкретным примером, представленным на рис. 1.2.



Рис. 1.2. Иллюстрация устойчивой и неустойчивой сортировки

В этом примере при устойчивой сортировке взаимное расположение равных элементов с ключом *1* и дополнительными полями "*a", "b", "c"* осталось прежним: элемент с полем *"a"*, затем – с *"b"*, затем – с *"c"*. При неустойчивой сортировке взаимное расположение равных элементов с ключом *1* и дополнительными полями "*a", "b", "c"* изменилось.

4. *Естественность поведения* – эффективность метода при обработке уже отсортированных, или частично отсортированных данных. Алгоритм ведет себя естественно, если учитывает эту характеристику входной последовательности и работает лучше.

Далее рассмотрим улучшенный алгоритм одного из элементарных алгоритмов сортировки, а именно, сортировки выбором, которым является так называемая пирамидальная сортировка.

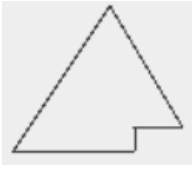
1.2. Алгоритм пирамидальной сортировки

Улучшением сортировки выбором является так называемая пирамидальная сортировка. В основу этого метода положена особая организация данных в массиве, называемая пирамидой.

Пирамида (Heap) – это бинарное дерево высоты *k*, удовлетворяющая следующим требованиям:

1) узлами дерева являются элементы массива;

2) дерево является сбалансированным, т.е. все листья имеют глубину *k* или *k – 1*, при этом уровень *k – 1* полностью заполнен, а уровень *k* заполнен слева направо, т.е форма пирамиды имеет приблизительно такой вид:



3) выполняется "свойство пирамиды": каждый элемент меньше либо равен родителю, а корнем дерева является максимальный элемент массива.

На рис. 1.3 приведен пример пирамиды, имеющей глубину 4.

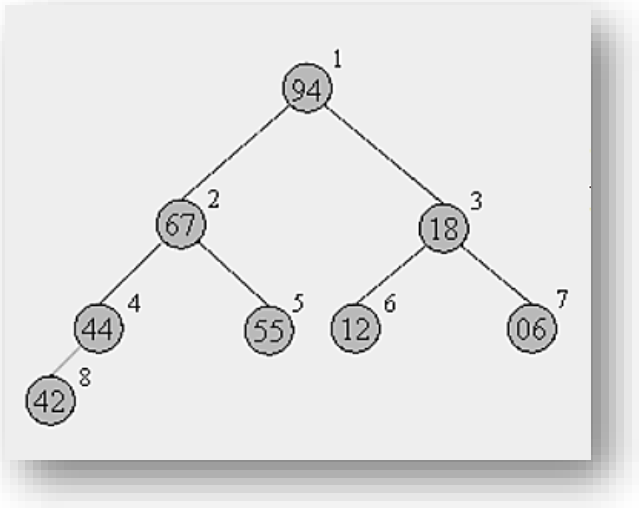


Рис. 1.3. Пример пирамиды

Для представления пирамиды не требуется создавать специальную структуру данных – ее можно хранить прямо в сортируемом массиве. Соответствие между геометрической структурой пирамиды как дерева и массивом устанавливается по следующей схеме: в *a[0]* хранится корень дерева, левый и правый сыновья элемента *a[i]* хранятся в *a[2i+1]* и *a[2i+2]* соответственно.

Для массива, хранящего в себе пирамиду, выполняется следующее характеристическое свойство: $a[i] \geq a[2i+1]$ и $a[i] \geq a[2i+2]$.

Плюсы такого хранения пирамиды очевидны:

- никаких дополнительных переменных, нужно лишь понимать схему;
- узлы хранятся от вершины и далее вниз, уровень за уровнем;
- узлы одного уровня хранятся в массиве слева направо.

Запишем в виде массива пирамиду, изображенную на рис. 1.3. Слева - направо, сверху - вниз: 94, 67, 18, 44, 55, 12, 06, 42. На рисунке место элемента пирамиды в массиве обозначено цифрой вверху и справа от него.

Восстановить пирамиду из массива как геометрический объект легко – достаточно вспомнить схему хранения и нарисовать, начиная от корня.

Пирамидальная сортировка включает две фазы: приведение исходного массива к виду пирамиды (построение пирамиды) и собственно сортировка. Рассмотрим эти фазы подробнее.

Фаза построения пирамиды. Начинать построение пирамиды можно с *a[l]...a[n – 1]*, где $l = n \ div \ 2$. Эта часть массива удовлетворяет свойству пирамиды, так как не существует индексов *i* и *j*, таких, что $i = 2l+1$ и $j = 2l+2$, просто потому, что такие *i* и *j* находятся за границей массива.

Следует заметить, что неправильно говорить о том, что *a[l] ... a[n – 1]* является пирамидой как самостоятельный массив. Это, вообще говоря, не верно: его элементы могут быть любыми. Свойство пирамиды сохраняется лишь в рамках исходного, основного массива *a[0] ... a[n – 1]*.

Далее будем расширять часть массива, добавляя по одному элементу за шаг. Следующий элемент на каждом шаге добавления – тот, который стоит перед уже готовой частью.

Чтобы при добавлении элемента сохранялась пирамидальность, будем использовать следующую процедуру расширения пирамиды *a[k+1]...a[n – 1]* влево на новый элемент *a[k]:*

1) смотрим на потомков *a[k]* слева и справа (если он существует) – в массиве это *a[2k+1*] и a*[2k+2]* и выбираем наибольшего из них.

2) если этот элемент больше *a[k]* – меняем его с *a[k]* местами и идем к шагу 1, имея в виду новое положение *a[k]* в массиве. Иначе конец процедуры.

Таким образом, новый элемент *a[k]* как бы "просеивается" сквозь пирамиду.

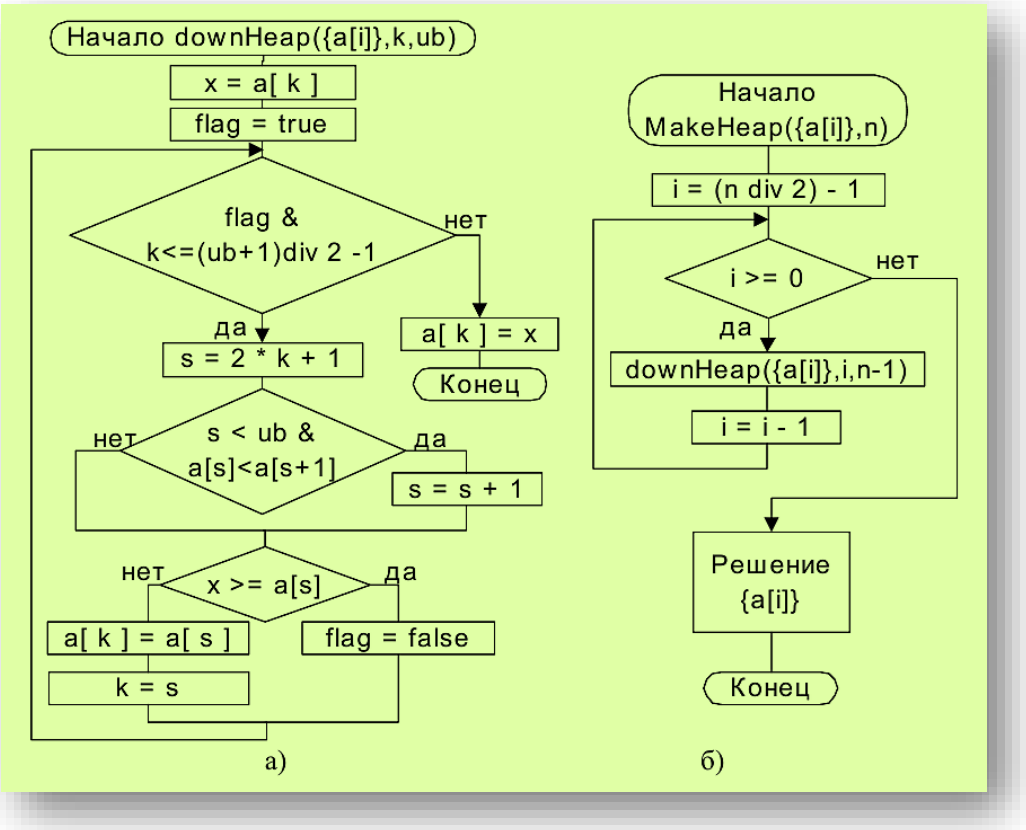


Рис. 1.4. Блок-схемы процедур приведения массива к пирамидальному виду:
а) процедура "просеивания" элемента сквозь пирамиду;
б) алгоритм построения пирамиды

На рис. 1.4 а) приведена блок-схема процедуры просеивания элемента сквозь пирамиду (*downHeap*). Входными параметрами этой процедуры являются: *{a[i]}* – преобразуемый массив, *k* – индекс просеиваемого элемента, *ub* – индекс последнего элемента массива.

Применяя процедуру просеивания к элементам *a[l-1], a[l-2],…, a[0]* получим пирамиду. На рис. 1.4 б) приведена блок-схема алгоритма построения пирамиды (*MakeHeap*).

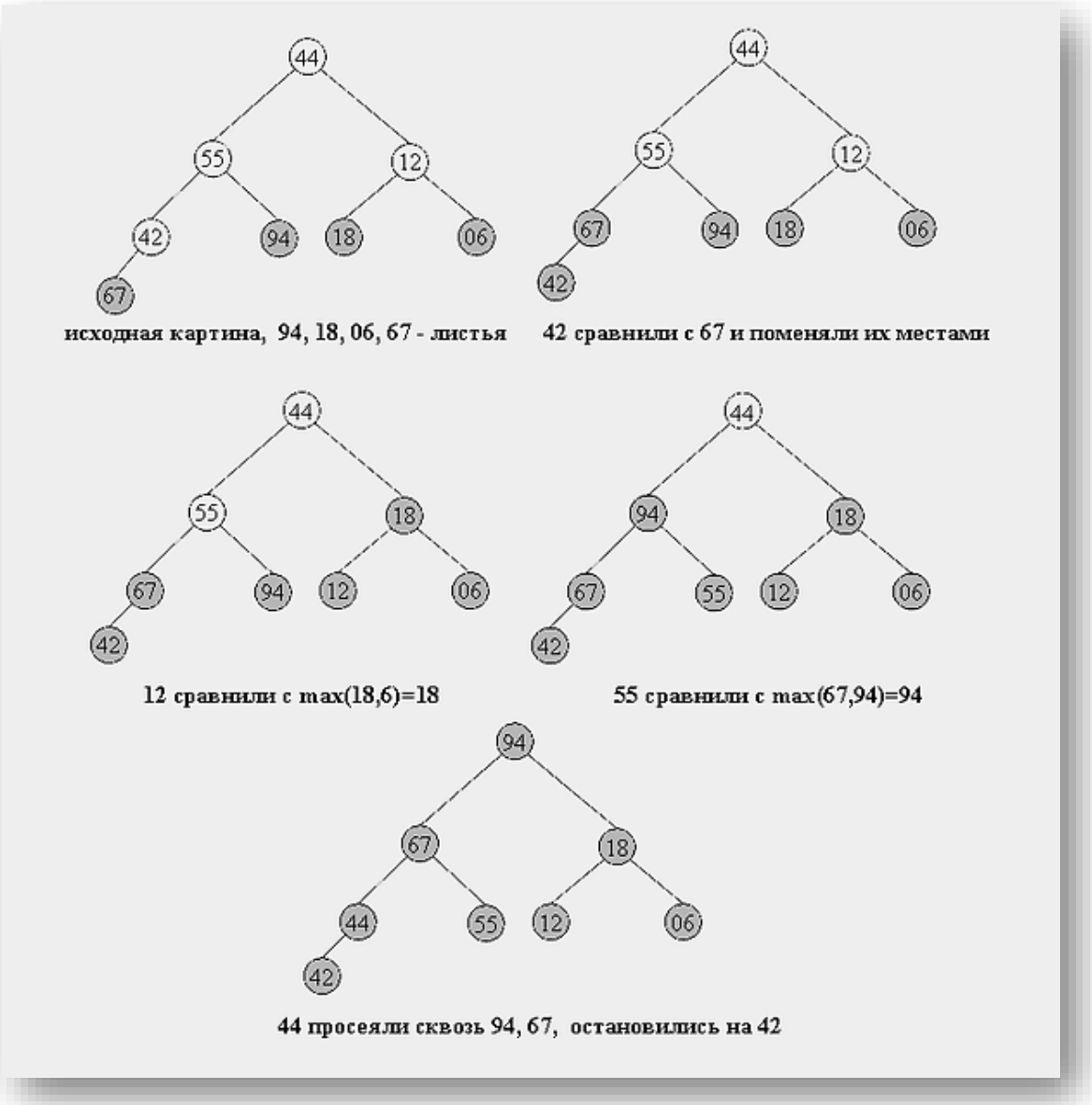


Рис. 1.5. Иллюстрация построения пирамиды

Рассмотрим пример построения пирамиды из исходного массива: 44, 55, 12, 42, 94, 18, 6, 67. В геометрической интерпретации элементы a[n div 2] ... a[n-1] (94, 18, 6, 67) является листьями в бинарном дереве. Один за другим остальные элементы продвигаются на свои места, и так, пока не будет построена вся пирамида. На рис. 1.5 изображен процесс построения пирамиды. Неготовая часть пирамиды (начало массива) окрашена в белый цвет, удовлетворяющий свойству пирамиды конец массива – в темный.

Фаза сортировки. Как видно из свойств пирамиды, в корне всегда находится максимальный элемент. Отсюда вытекает алгоритм сортировки:

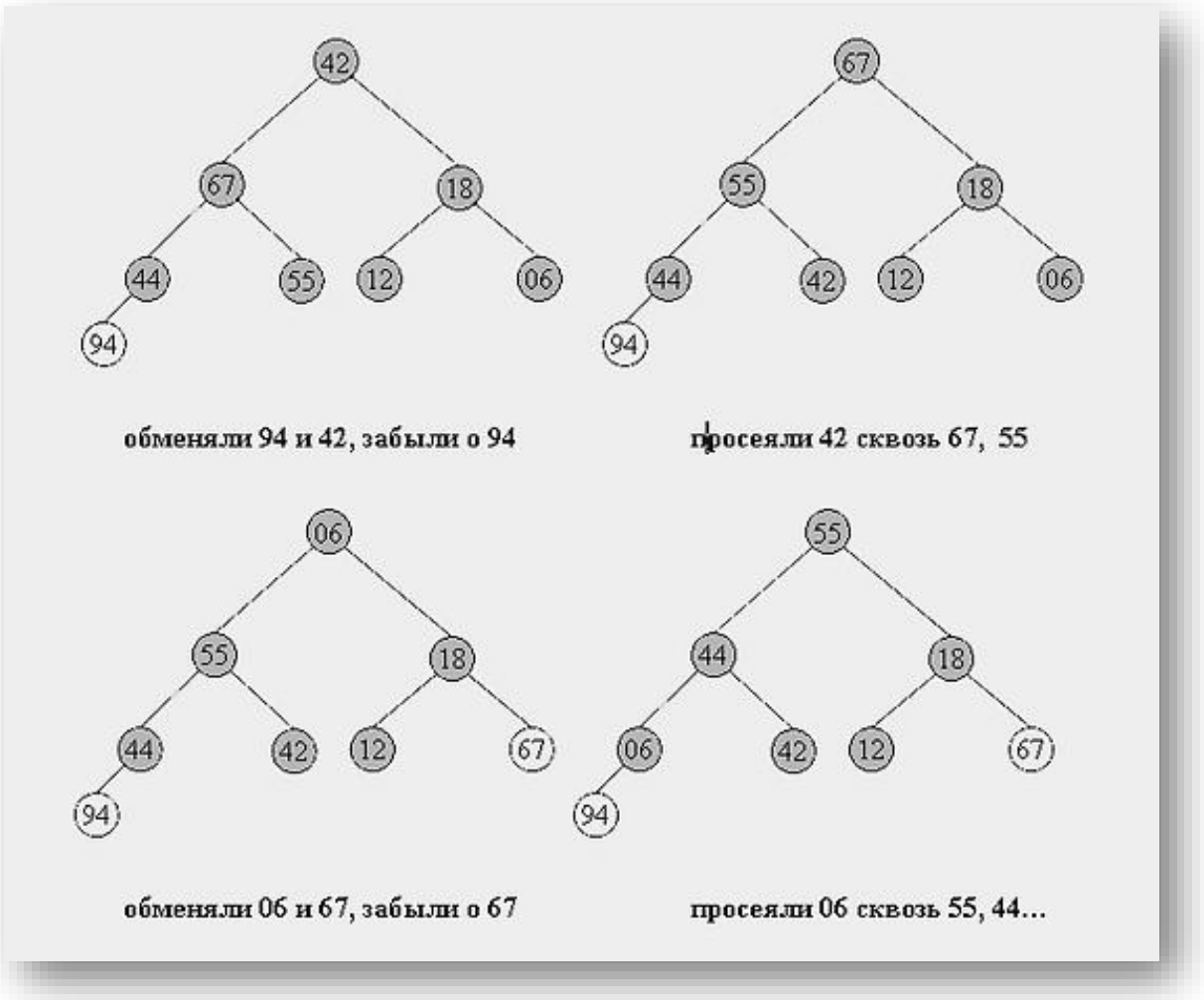


Рис. 1.6. Иллюстрация алгоритма пирамидальной сортировки

1) берем верхний элемент пирамиды a[0]...a[n-1] (первый в массиве) и меняем с последним местами. Теперь "забываем" об этом элементе и далее рассматриваем массив a[0]...a[n – 2]. Для превращения его в пирамиду достаточно просеять лишь новый первый элемент;

2) повторяем шаг 1, пока обрабатываемая часть массива не уменьшится до одного элемента.

На рис. 1.6 приведен пример иллюстрирующий этот алгоритм.

Очевидно, в конец массива каждый раз попадает максимальный элемент из текущей пирамиды, поэтому в правой части постепенно возникает упорядоченная последовательность.

На рис. 1.7 приведена блок-схема алгоритма пирамидальной сортировки (*HeapSort*).

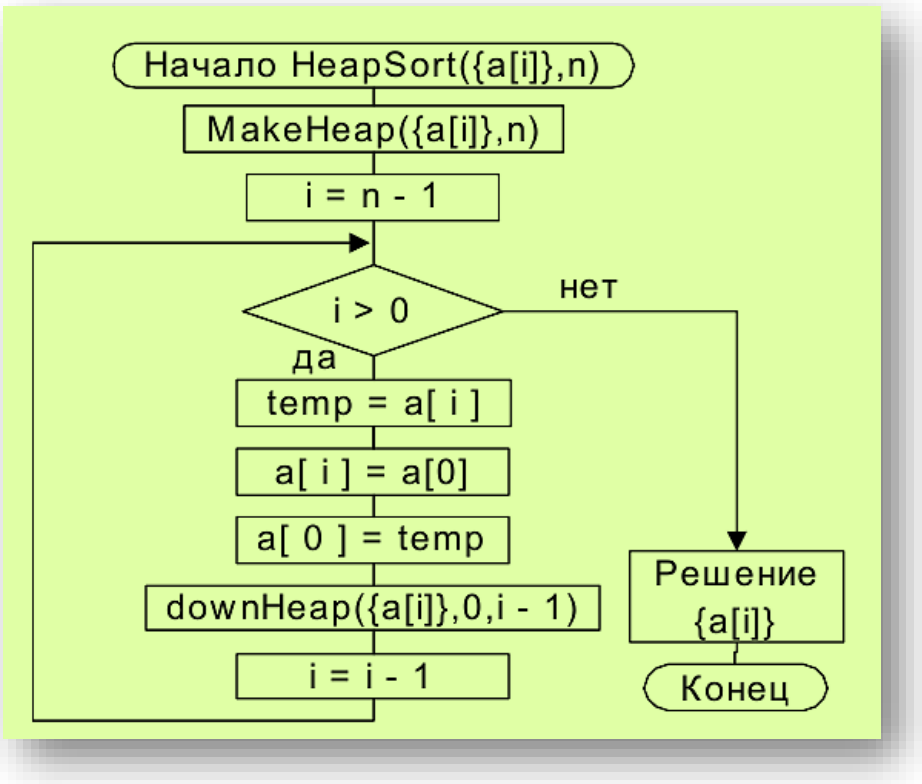


Рис. 1.7. Блок-схема алгоритма пирамидальной сортировки

Каково быстродействие получившегося алгоритма? Учитывая, что высота пирамиды не более $log_2 n$, процедура *downHeap* требует времени исполнения порядка $O(log\ n)$. Построение пирамиды занимает порядка $O(n\ log\ n)$ операций, причем более точная оценка дает даже $O(n)$ за счет того, что реальное время выполнения *downHeap* зависит от высоты уже созданной части пирамиды.

Вторая фаза занимает $O(n\ log\ n)$ времени: $O(n)$ раз берется максимум и происходит просеивание бывшего последнего элемента. Плюсом является стабильность метода: среднее число пересылок ($n\ log\ n$)/2, и отклонения от этого значения сравнительно малы.

Пирамидальная сортировка не использует дополнительной памяти. Метод не является устойчивым: по ходу работы массив так "перетряхивается", что исходный порядок элементов может измениться случайным образом. Поведение неестественно: частичная упорядоченность массива никак не учитывается.

2. Программная реализация

Программа "Пирамидальная сортировка" представляет собой приложение, разработанное с использованием языка C# и Windows Forms в среде Visual Studio. Она реализует алгоритм пирамидальной сортировки, который позволяет упорядочить элементы массива в порядке возрастания или убывания.

2.1. Основные функциональные возможности программы:

1. Генерация массива:

- Пользователь может ввести данные для создания массива, который нужно отсортировать.
- Возможность настройки генерируемого массива как по количеству элементов, так и по их максимальному и минимальному значению.
- Проверка корректности ввода данных.

2. Пирамидальная сортировка:

- Реализация алгоритма пирамидальной сортировки для упорядочивания списка.
- Показ промежуточных шагов сортировки для наглядности, а также настройка скорости отображения элементов.

3. Вывод отсортированного массива:

- Отображение отсортированного массива элементов с помощью графика.

4. Дополнительные функции:

- Возможность работы с большими объемами данных, обеспечивая эффективность и скорость сортировки.
- Предоставление пользователю информации о времени выполнения сортировки.
- Дополнительный элемент визуализации, который позволяет сравнить скорость работы пирамидальной сортировки с другими алгоритмами.

Интерфейс программы построен с использованием Windows Forms, предоставляющей удобные элементы управления для ввода данных массива, отображения результатов сортировки и взаимодействия с пользователем. Текстовое поле используется для ввода данных генерируемого массива, кнопка "Сортировать" для запуска алгоритма сортировки, а также кнопка "Рассчитать время" для вывода времени работы алгоритма.

Программа реализует пирамидальный алгоритм сортировки, который работает на основе структуры данных "куча" (heap). Алгоритм обрабатывает список элементов, формируя кучу и последовательно извлекая наибольший элемент. Каждое извлечение будет приводить к правильной упорядоченности списка, пока весь список не будет отсортирован.

Цель программы - предоставить пользователю удобный способ сортировки массива элементов с помощью пирамидальной сортировки и дать возможность визуализировать процесс сортировки для лучшего понимания алгоритма.

2.2. Описание программной оболочки

Программное обеспечение было разработано с использованием алгоритма пирамидальной сортировки. Ниже описаны шаги этого алгоритма (рис. 2.1), а также метод "heapify" для работы с кучей.

1. Построение пирамиды (Heap Construction):

- На вход алгоритма подается неотсортированный массив.
- Алгоритм проходит по элементам массива справа налево, начиная с последнего элемента, и применяет процедуру "heapify" для каждого элемента.
- Процедура "heapify" позволяет сформировать структуру данных "куча", где каждый элемент массива располагается в соответствующей позиции в виде дерева.

2. Сортировка пирамиды (Heap Sort):

После построения пирамиды, сама сортировка выполняется следующим образом:

- Максимальный элемент массива находится в корне кучи.
- Этот элемент меняется местами с последним элементом массива.
- После этого, размер кучи уменьшается на 1 (исключая уже отсортированный элемент).
- Затем, для нового корня кучи, применяется процедура "heapify" для поддержания свойств кучи.

Процесс повторяется до тех пор, пока весь массив не будет отсортирован.

```
Ссылок: 2
public static void Sorting(int[] arr) // Пирамидальная сортировка
{
    int n = arr.Length;

    // Построение кучи (перегруппируем массив)
    for (int i = n / 2 - 1; i >= 0; i--)
        heapify(arr, n, i);

    // Один за другим извлекаем элементы из кучи
    for (int i = n - 1; i >= 0; i--)
    {
        // Перемещаем текущий корень в конец
        int temp = arr[0];
        arr[0] = arr[i];
        arr[i] = temp;

        // вызываем процедуру heapify на уменьшенной куче
        heapify(arr, i, 0);
    }
}
```

Рис. 2.1. Основной метод пирамидальной сортировки

3. Метод heapify

Этот метод (рис. 2.2.) принимает на вход массив и индекс элемента, который нужно проверить и, при необходимости, перестроить поддерево в соответствии со свойствами кучи:

- Если элемент имеет потомков (левого и правого), метод сравнивает их со значением текущего элемента.
- Если один из потомков больше текущего элемента, то они меняются местами, и процесс рекурсивно повторяется для поддерева.

Алгоритм пирамидальной сортировки и метод heapify обеспечивают эффективную сортировку элементов в массиве, где каждая операция преобразования и перестройки выполняется на основе структуры данных "куча".

```
// Процедура для преобразования в двоичную кучу поддерева с корневым узлом i, что является
// индексом в arr[]. n - размер кучи

Ссылок: 3
public static void heapify(int[] arr, int n, int i)
{
    int largest = i;
    // Инициализируем наибольший элемент как корень
    int l = 2 * i + 1; // left = 2*i + 1
    int r = 2 * i + 2; // right = 2*i + 2

    // Если левый дочерний элемент больше корня
    if (l < n && arr[l] > arr[largest])
        largest = l;

    // Если правый дочерний элемент больше, чем самый большой элемент на данный момент
    if (r < n && arr[r] > arr[largest])
        largest = r;

    // Если самый большой элемент не корень
    if (largest != i)
    {
        int swap = arr[i];
        arr[i] = arr[largest];
        arr[largest] = swap;

        // Рекурсивно преобразуем в двоичную кучу затронутое поддерево
        heapify(arr, n, largest);
    }
}
```

Рис 2.2. Специальный метод для работы с кучей

2.3. Интерфейс приложения

Приложение имеет следующие поля:

1. Поле "Количество элементов":

   - возможность ввести количество элементов генерируемого массива.

2. Поле "Минимальное значение":

   - возможность ввести минимальное значение элементов в генерируемом массиве.

3. Поле "Максимальное значение":

   - возможность ввести максимальное значение элементов в генерируемом массиве.

4. Поле "Время обновления (2-й график)":

   - возможность ввести время обновления для второго графика, который отображает скорость работы пирамидальной сортировки.

5. Поле "Время обновления (3-й график)":

   - возможность ввести время обновления для третьего графика, который показывает скорость работы пирамидальной сортировки сравнительно с другими типами сортировок.

Приложение также содержит следующие кнопки:

1. Кнопка "Генерация массива":

   - при нажатии на эту кнопку происходит генерация массива с указанными параметрами (количество элементов, минимальное и максимальное значения).

2. Кнопка "Построить пирамиду":

   - при нажатии на эту кнопку выполняется построение пирамиды на основе сгенерированного массива.

3. Кнопка "Отсортировать пирамиду":

   - при нажатии на эту кнопку происходит сортировка пирамиды, полученной после построения.

4. Кнопка "Рассчитать время":

   - при нажатии на эту кнопку запускается процесс расчёта времени работы пирамидальной сортировки.

5. Кнопка "Перейти к графику сравнения скорости работы алгоритма":

   - при нажатии на эту кнопку открывается отдельный график, на котором отображается скорость работы пирамидальной сортировки сравнительно с другими типами сортировок.

Таким образом, пользователь может вводить необходимые параметры через соответствующие поля и использовать кнопки для выполнения определенных операций, таких как генерация массива, построение и сортировка пирамиды, расчёт времени работы и просмотр графиков для анализа результатов.

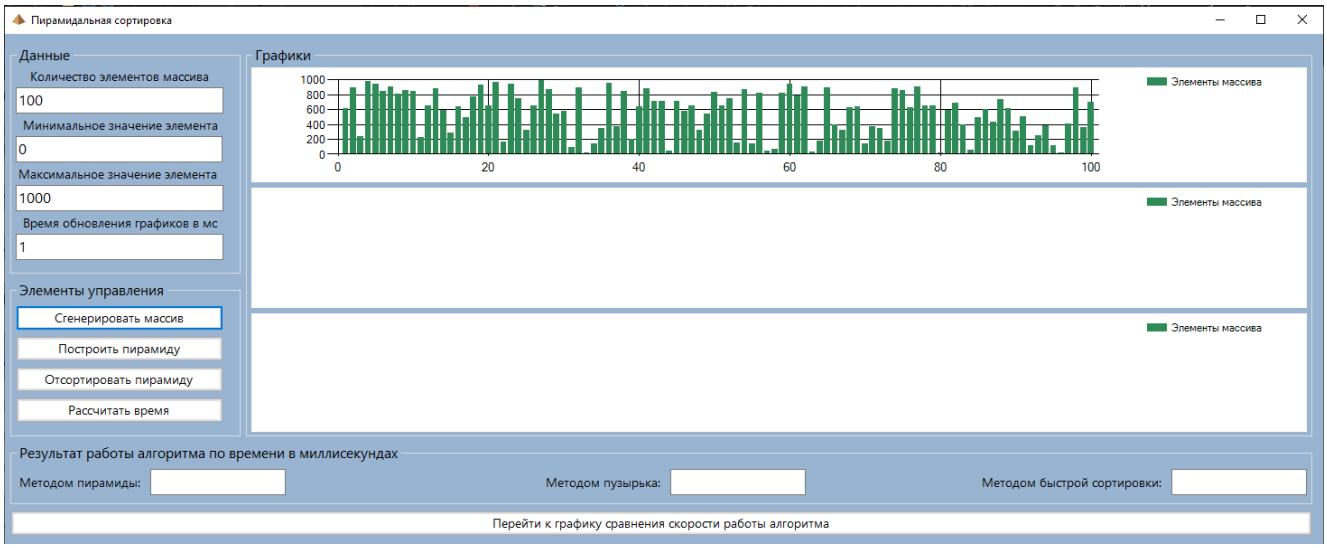


Рис. 2.3. Этап 1 – создание массива по заданным параметрам

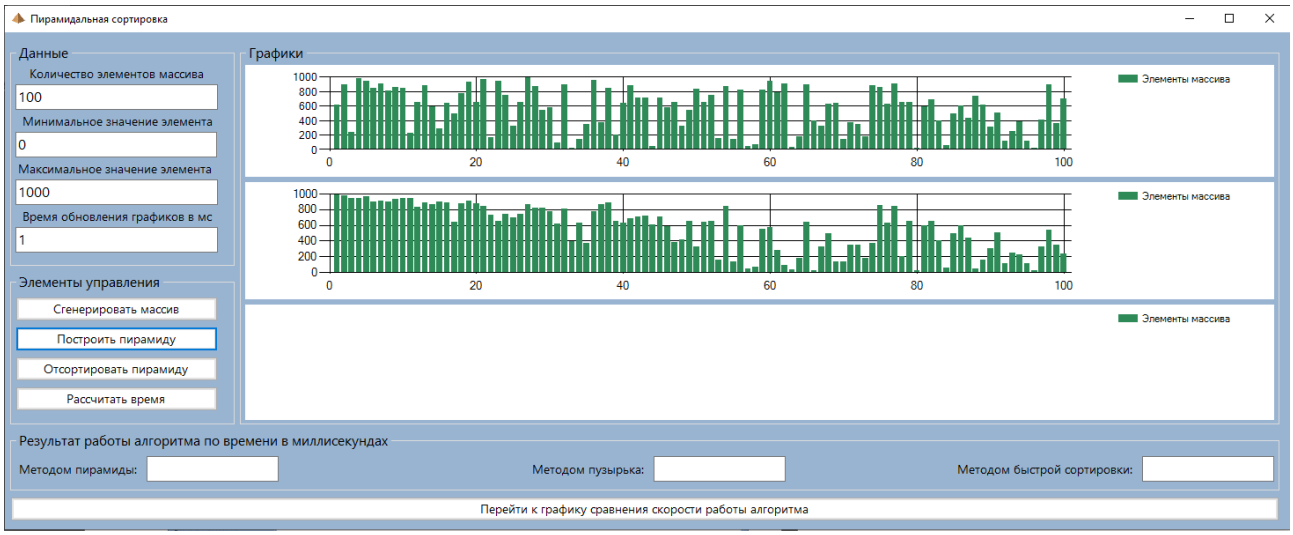


Рис. 2.4. Этап 2 – создание пирамиды по введённому массиву

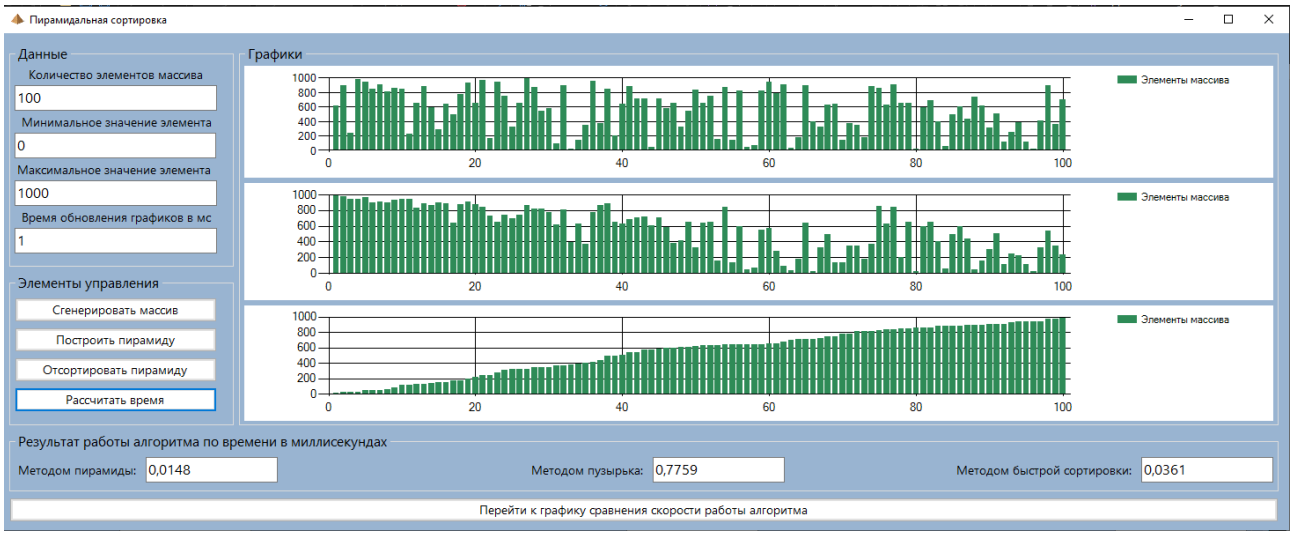


Рис. 2.5. Этап 3 – сортировка пирамиды и вывод результата работы по времени

При нажатии на кнопку "Перейти к графику сравнения скорости работы алгоритма" в программе появляется отдельное окно (рис. 2.6.), которое содержит следующие поля:

1. ***Поле "Увеличение размера сортируемого массива на шаг".*** В этом поле можно указать значение шага, на которое будет увеличиваться размер сортируемого массива при каждой итерации. Например, если указан шаг равный 100, то при каждом запуске алгоритма размер массива будет увеличиваться на 100 элементов.

2. ***Поле "Интервал отрисовки".*** В данном поле можно задать интервал времени в миллисекундах, с которым будет обновляться график сравнения скорости работы алгоритма. Например, если указан интервал отрисовки равный 1000, то график будет обновляться каждую секунду.

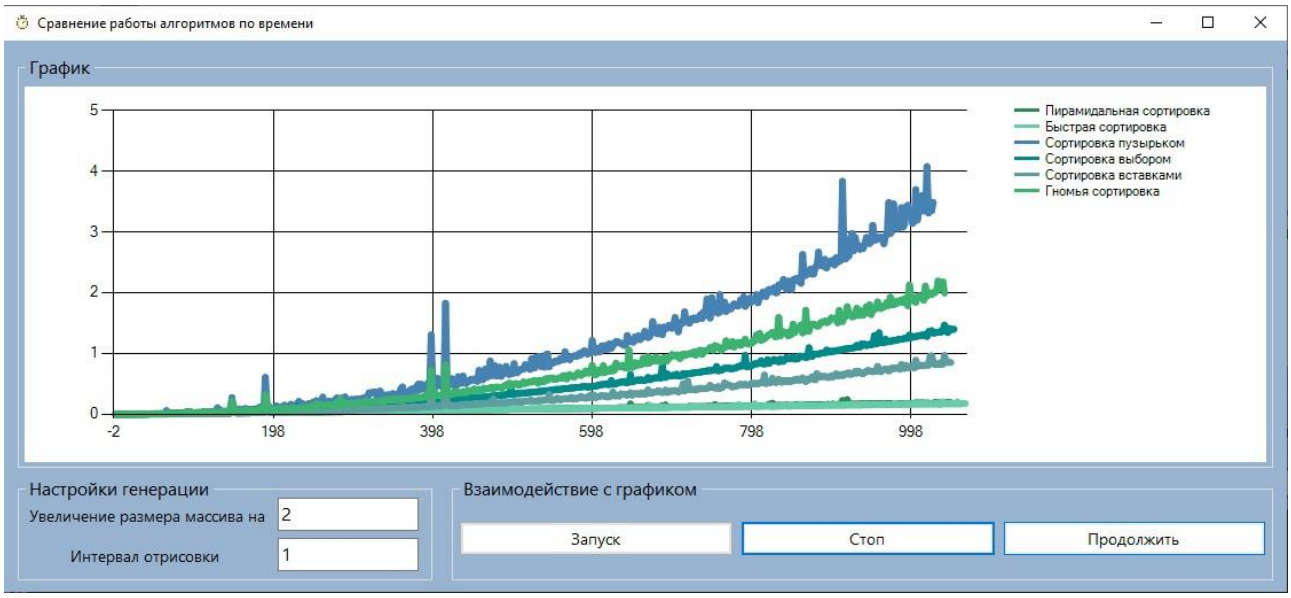


Рис. 2.6. Сравнение времени (мс) работы алгоритмов в зависимости от количества элементов сортируемого массива

Кроме того, в окне присутствуют следующие кнопки:

1. ***Кнопка "Запуск".*** При нажатии на эту кнопку начинается выполнение алгоритма с заданными параметрами (шагом сортируемого массива и интервалом отрисовки).

2. ***Кнопка "Стоп".*** Нажатие на данную кнопку прерывает выполнение алгоритма и останавливает обновление графика.

3. ***Кнопка "Продолжить".*** Если график был остановлен с помощью кнопки "Стоп", нажатие на кнопку "Продолжить" возобновляет его выполнение с момента остановки и продолжает обновление графика.

Таким образом, пользователь может настроить параметры работы графика, запустить его выполнение, остановить или возобновить работу, а также наблюдать динамическое изменение сравнения скорости работы алгоритмов.

Примечание. Из-за погрешности работы таймера график имеет некоторые всплески по времени работы алгоритма.

Важно отметить, что при расчете времени работы алгоритмов как в основной программе, так и в отдельном окне для сравнения их скорости, используются отдельные потоки. Это позволяет ускорить работу приложения и избежать его зависания.

Расчет времени работы алгоритмов в основной программе выполняется в отдельном потоке, отделенном от графического интерфейса пользователя. Таким образом, пользователь может продолжать взаимодействие с приложением, в то время как алгоритмы выполняются в фоновом режиме. Это избавляет приложение от ощутимой задержки или блокировки во время выполнения длительных операций.

Аналогично, когда пользователь запускает отдельное окно для сравнения времени работы алгоритмов, процесс расчета также выполняется в отдельном потоке. Это позволяет пользователю продолжать работу с другими функциями приложения, в то время как происходит расчет времени работы различных алгоритмов. Отдельный поток позволяет обеспечить плавность интерфейса и избежать его зависания.

Такой подход с использованием отдельных потоков для расчета времени работы алгоритмов обеспечивает более отзывчивую и плавную работу программы, поскольку пользовательский интерфейс остается отзывчивым даже во время выполнения вычислительно интенсивных операций.

Алгоритм производит сравнение времени работы следующих сортировок: пирамидальная, быстрая, сортировка пузырьком, гномья, сортировка выбором и сортировка вставками.

1. Пирамидальная сортировка.

- алгоритм основан на структуре данных "куча" (или пирамида);
- сначала формируется куча из исходного массива данных;
- затем извлекается наибольший (или наименьший) элемент и помещается в конец массива;

- после этого куча перестраивается, чтобы удовлетворять условиям кучи;
- процесс повторяется до полной сортировки массива;
- пирамидальная сортировка имеет сложность O(n log n).

2. Быстрая сортировка.

- алгоритм основан на принципе "разделяй и властвуй";
- выбирается опорный элемент из массива (например, первый или случайный);
- массив разбивается на две части: элементы, меньшие опорного, и элементы, большие опорного;
- рекурсивно применяется быстрая сортировка к каждой из частей.
- затем объединяются отсортированные части;
- быстрая сортировка имеет среднюю сложность O(n log n), но в худшем случае может быть $O(n^2)$.

3. Сортировка пузырьком.

- алгоритм сравнивает пары соседних элементов и меняет их местами, если они находятся в неправильном порядке;
- процесс повторяется до полной сортировки массива;
- сортировка пузырьком имеет сложность $O(n^2)$.

4. Гномья сортировка.

- алгоритм работает, перемещая элементы назад, пока не будет найдено правильное место;
- если элементы находятся в правильном порядке, алгоритм перемещается вперед;
- гномья сортировка имеет сложность $O(n^2)$, но может быть эффективной на уже почти отсортированных данных.

5. Сортировка выбором.

- алгоритм находит минимальный элемент и помещает его в начало массива;

- затем он ищет следующий минимальный элемент и помещает его на следующую позицию и т.д.

Программа также включает в себя свой установщик, который предоставляет пользователю возможность выбрать место установки файлов программы и создать ярлык приложения на рабочем столе.

Установщик программы предоставляет пользователю удобный интерфейс, позволяющий выбрать директорию, в которую будут скопированы файлы программы. Пользователь может выбрать существующую директорию или создать новую.

После выбора расположения файлов программа копирует необходимые файлы в указанную директорию. Это включает исполняемый файл программы, библиотеки, ресурсы и другие необходимые компоненты.

Кроме того, установщик позволяет пользователю создать ярлык приложения на рабочем столе. Это обеспечивает удобный доступ к программе через ярлык, что позволяет быстро запускать приложение.

Установщик программы обеспечивает понятный и интуитивно понятный процесс установки, который позволяет пользователям выбирать опции, соответствующие их предпочтениям. Это способствует удобству использования программы и обеспечивает более гибкую настройку расположения файлов программы в системе.

Заключение

В данном учебно-методическом пособии представлено приложение, реализующее алгоритм пирамидальной сортировки с визуализацией всего вычислительного процесса, что улучшает понимание самого алгоритма.

Интерфейс программы предоставляет удобные элементы управления для ввода данных массива, отображения результатов сортировки и взаимодействия с пользователем.

Пользователь может настраивать параметры в соответствии со своими потребностями при изучении пирамидальной сортировки, проводить вычислительные эксперименты с различными размерами сортируемых массивов.

Приложение позволяет оценить эффективность пирамидальной сортировки и сравнить ее время работы с временем работы следующих алгоритмов: сортировка выбором, вставками, пузырьковая сортировка, гномья и быстрая сортировки.

При запуске алгоритма с заданными параметрами, пользователи могут наблюдать изменения на графике, который демонстрирует время работы пирамидальной и других сортировок одновременно в зависимости от размера сортируемого массива. Это визуальное представление позволяет лучше понять, как работает алгоритм и как его эффективность зависит от размера данных.

Таким образом, данное приложение создано с целью облегчить понимание и усвоение алгоритма пирамидальной сортировки, предоставляя возможность пользователям экспериментировать с различными параметрами и визуально наблюдать результаты работы алгоритма.

Разработанное приложение «Пирамидальная сортировка» используется в учебном процессе при чтении лекций и проведении лабораторных и практических занятий по дисциплине «Структуры и алгоритмы обработки данных» для студентов направлений подготовки 09.03.01 "Информатика и вычислительная техника" и 09.03.04 «Программная инженерия».

Литература:

1. Рысин, М. Л. Введение в структуры и алгоритмы обработки данных : учебное пособие / М. Л. Рысин, М. В. Сартаков, М. Б. Туманова. - Москва : РТУ МИРЭА, 2022 - Часть 1 : Сложность алгоритмов. Сортировки. Линейные структуры данных. Поиск в таблице - 2022. - 110 с. - ISBN 978-5-7339-1612-5. - Текст : электронный // Лань : электронно-библиотечная система. - URL: https://e.lanbook.com/book/256592 (дата обращения: 30.05.2023). - Режим доступа: для авториз. пользователей.

2.. Варфоломеева, Т. Н. Структуры данных и основные алгоритмы их обработки : учебное пособие / Т. Н. Варфоломеева. - Москва : ФЛИНТА, 2017. - 159 с. - ISBN 978-5-9765-3691-3. - Текст : электронный // Лань : электронно-библиотечная система. - URL: https://e.lanbook.com/book/104903 (дата обращения: 30.05.2023). - Режим доступа: для авториз. пользователей.

3. Кнут Д. Искусство программирования [Текст] / Д. Э. Кнут ; ред. Ю.В. Козаченко. - Испр. и доп. изд. - М.; СПб.; К. : Вильямс, 2000 - 2001. - (Классический труд). - Тит. л. рус., англ. - Т. 1 : Основные алгоритмы / пер.: С. Г. Тригуб, Ю. Г. Гордиенко, И. В. Красиков. - 3-е изд. - 2001. - 712 с.

4. Кнут Д. Искусство программирования [Текст] / Д. Э. Кнут ; ред. Ю.В. Козаченко. - Испр. и доп. изд. - М.; СПб.; К. : Вильямс, 2000 - 2001. - (Классический труд). - Тит. л. рус., англ. - Т. 2 : Получисленные алгоритмы / пер.: В.Т. Тертышный, Л.Ф. Козаченко, И.В. Красиков. - 3-е изд. - 2001. – 828 с.

5. Кнут Д. Искусство программирования [Текст] / Д. Э. Кнут ; ред. Ю.В. Козаченко. - Испр. и доп. изд. - М.; СПб.; К. : Вильямс, 2000 - 2001. - (Классический труд). - Тит. л. рус., англ. - Т. 3 : Сортировка и поиск / пер.: В. Т. Тертышный, И. В. Красиков. - 2-е изд. - 2000. - 822 с.